



Database Constraints & Integrity

Ensuring Accurate, Reliable & Consistent Data

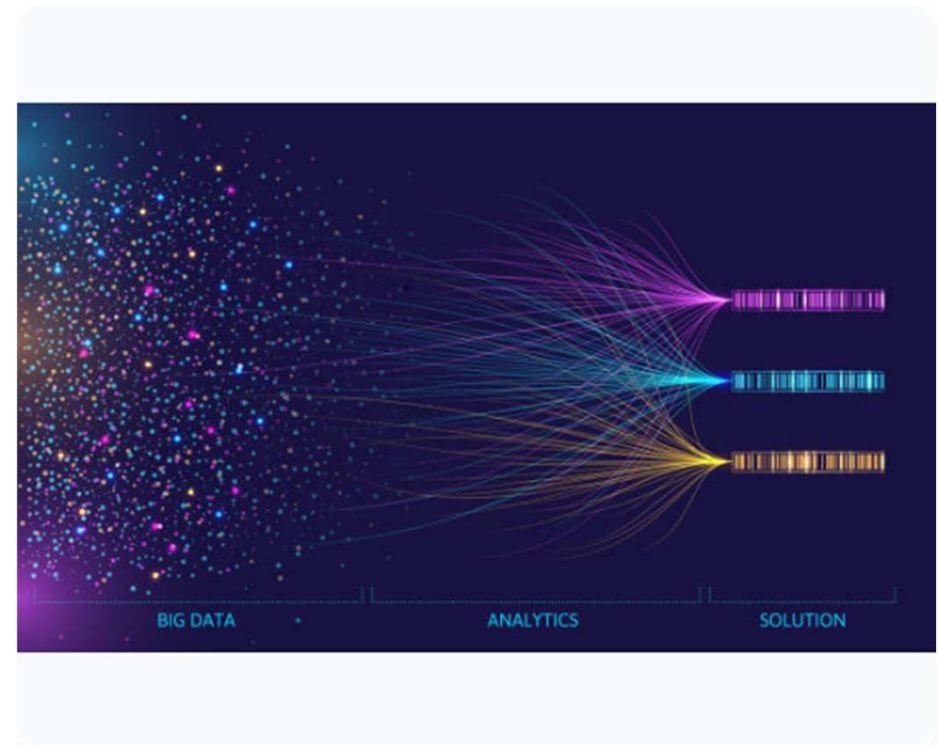


Motivation: A World Without Constraints

Imagine the database of an **Online University** with no rules. We might see:

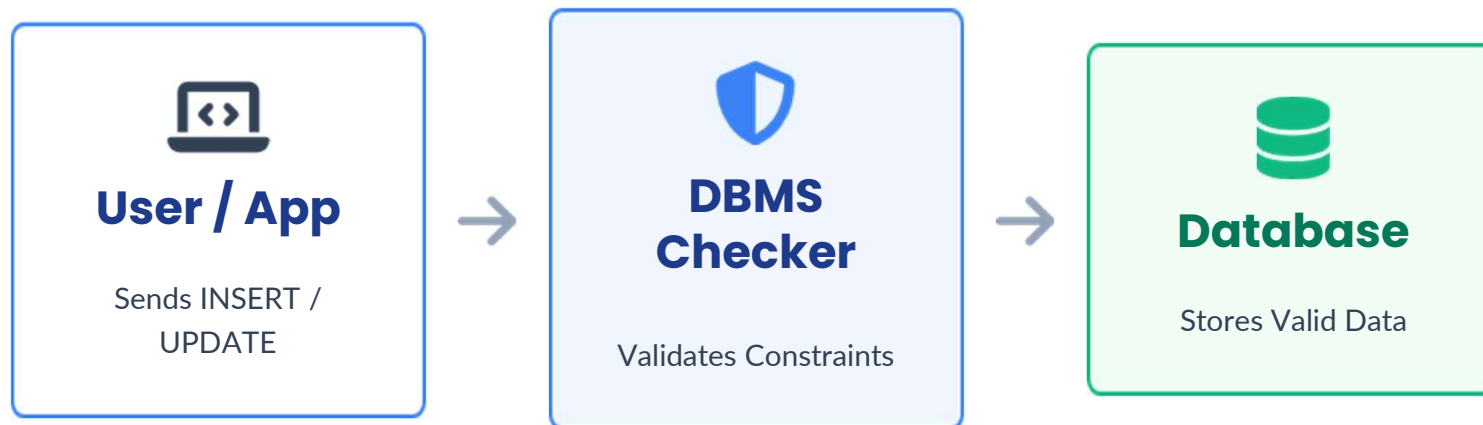
- ⚠ Student ID is duplicated
- ⚠ Student Name is NULL
- ⚠ Enrolled Department doesn't exist
- ⚠ Age = -5

Discussion: Can we trust this database?



What are Constraints?

Constraints are strict, database-level rules enforced by the DBMS to guarantee **data validation**, **data consistency**, and adherence to **business rules**.





Types of Core Constraints

Constraint Type	Definition & Purpose	Real-World Example
NOT NULL	Ensures a column cannot contain a NULL (empty) value.	A user must provide a First Name.
UNIQUE	Guarantees all values in a column are entirely distinct.	No two users can have the same Email.
PRIMARY KEY	Uniquely identifies each record (NOT NULL + UNIQUE).	National ID, Student Roll Number.
FOREIGN KEY	Links two tables, ensuring referential integrity.	An Order must belong to a valid Customer.
CHECK	Validates data against a specific condition/boolean expression.	Product Price must be > \$0.00.

NOT NULL Constraint

Hospital Patient Registration

A patient record is meaningless without a name. The NOT NULL constraint ensures required data is always provided.

VALID

```
INSERT INTO Patient VALUES (1, 'Alice', 25);
```

INVALID

```
INSERT INTO Patient VALUES (2, NULL, 30);
```

```
CREATE TABLE Patient ( PatientID INT PRIMARY KEY,  
PatientName VARCHAR(100) NOT NULL, Age INT );
```



UNIQUE Constraint

User Authentication

When users sign up for an app, their email addresses must be unique to prevent account duplication.

VALID

```
INSERT INTO Users VALUES (1, 'alice@gmail.com');
```

DUPLICATE REJECTED

```
INSERT INTO Users VALUES (2, 'alice@gmail.com');
```

```
CREATE TABLE Users ( UserID INT PRIMARY KEY, Email  
VARCHAR(100) UNIQUE );
```

PRIMARY KEY Constraint

- Uniquely identifies every record.
- Cannot be NULL.
- Only **one** Primary Key per table.

Real-world analogy: Your National ID Number or Fingerprint.

```
CREATE TABLE Student ( StudentID INT PRIMARY KEY,  
StudentName VARCHAR(100) );
```

Duplicate ID attempt fails:

```
INSERT INTO Student VALUES (101, 'John'); INSERT  
INTO Student VALUES (101, 'David');
```



Constraint Comparison

Feature	NOT NULL	UNIQUE	PRIMARY KEY
Allows NULL?	No	Yes (Varies by DBMS)	No
Allows Duplicates?	Yes	No	No
Allowed Per Table?	Many	Many	Only One

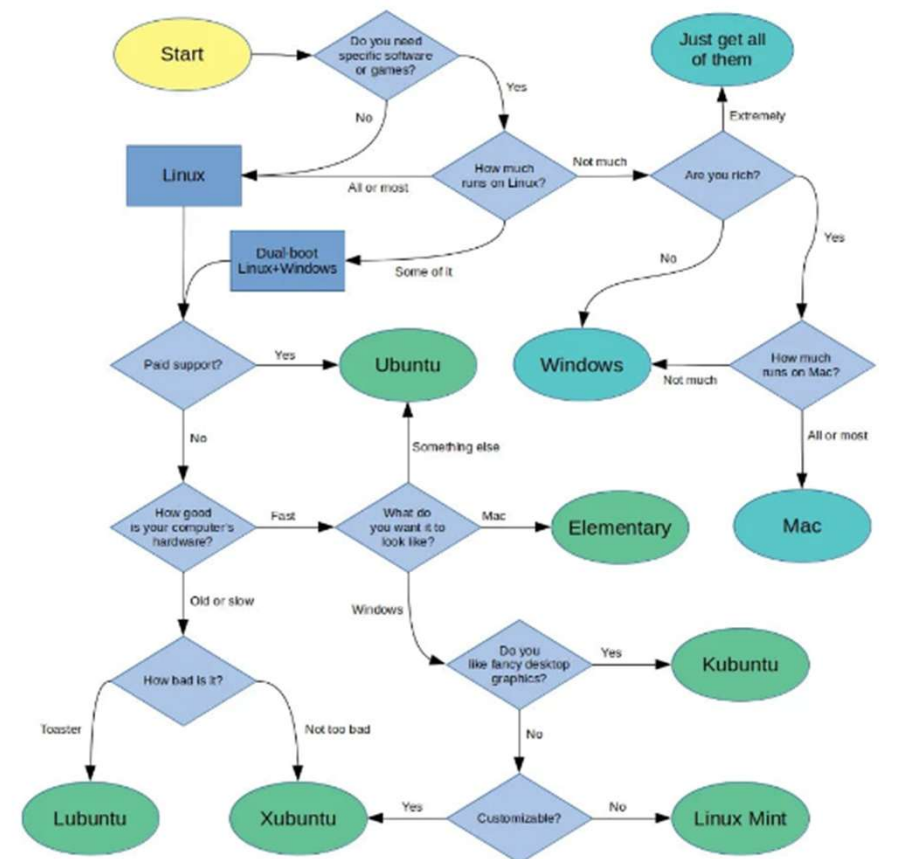
Note: While a table only has one Primary Key, that key can be "composite" (made of multiple columns).



FOREIGN KEY Constraint

Establishes a Parent-Child relationship between tables.

```
CREATE TABLE Department ( -- Parent
  DepartmentID INT PRIMARY KEY,
  DepartmentName VARCHAR(50) );
CREATE TABLE Student ( -- Child
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(100),
  DepartmentID INT,
  FOREIGN KEY (DepartmentID)
  REFERENCES Department(DepartmentID) );
```



Referential Integrity in Action

1. Populate Parent Table

```
INSERT INTO Department VALUES (1, 'CSE'), (2, 'EEE');
```

2. Insert Valid Child

✓ Student in Dept 1 (CSE)

```
INSERT INTO Student VALUES (101, 'Alice', 1);
```

3. Insert Invalid Child

What happens if we assign a student to Department 10?

✗ FAILS: Dept 10 doesn't exist

```
INSERT INTO Student VALUES (102, 'Bob', 10);
```

Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails.

Foreign Key Actions (ON DELETE / ON UPDATE)

What happens to Child records when the Parent record is deleted?

CASCADE

Automatically deletes/updates all matching child rows.

Example: Delete Dept → Delete all its Students.

SET NULL

Sets the Foreign Key column in child rows to NULL.

Example: Delete Dept → Students remain, DeptID is NULL.

RESTRICT

Prevents deletion of the parent if child rows exist.

(This is the Default behavior in MySQL).

NO ACTION

Standard SQL keyword. In MySQL, it is identical to RESTRICT (checked immediately).

CHECK Constraint

Validates data using a boolean condition. Added in MySQL 8.0.16.

- ✓ Age $\geq$ 18
- ✓ Salary > 0
- ✓ GPA BETWEEN 0 AND 4.0

✗ INVALID INSERT

```
INSERT INTO Employee VALUES (1, 15, 5000); --  
Fails: Age is under 18
```

```
CREATE TABLE Employee ( EmployeeID INT PRIMARY  
KEY, Age INT, CHECK (Age  $\geq$  18), Salary  
DECIMAL(10,2), CHECK (Salary > 0) );
```

Domain Constraints

A Domain Constraint ensures a column accepts only a predefined set of valid values.

Examples:

○ Gender: 'M', 'F', 'O'

☰ Status: 'Pending', 'Approved', 'Rejected'

```
CREATE TABLE Orders ( OrderID INT PRIMARY KEY,  
Status VARCHAR(20), CHECK (Status IN ( 'Pending',  
'Approved', 'Rejected' )) );
```

Putting It All Together

```
CREATE TABLE Student ( -- 1. PRIMARY KEY StudentID INT PRIMARY KEY, -- 2. NOT NULL
StudentName VARCHAR(100) NOT NULL, -- 3. UNIQUE Email VARCHAR(100) UNIQUE, -- 4. CHECK Age
INT CHECK (Age ≥ 18), DepartmentID INT, -- 5. FOREIGN KEY FOREIGN KEY (DepartmentID)
REFERENCES Department(DepartmentID) ON DELETE RESTRICT );
```

The Impact of Constraints

⚠ Without Constraints

ID	Name	Email
1	NULL	abc@gmail
1	John	abc@gmail
2	Mike	-

* Messy, unqueryable, breaks application logic.

With Constraints

ID	Name	Email
1	Alice	alice@univ.edu
2	John	john@univ.edu
3	Mike	mike@univ.edu

* Clean, reliable, ready for data analytics.



Class Activity: Map the Constraint

Which constraint should be applied to the following fields?

1 Student ID

.

→ PRIMARY KEY

2 Email Address

.

→ UNIQUE

3 Student Name

.

→ NOT NULL

4 Age > 18

.

→ CHECK

5 Department ID

.

→ FOREIGN KEY



Common Database Design Mistakes

- ✗ **Forgetting the PRIMARY KEY:** Leaves tables without row identity.
- ✗ **Missing UNIQUE on Emails/Usernames:** Causes duplicate accounts.
- ✗ **Allowing NULLs everywhere:** Leads to missing critical data in UI.
- ✗ **Skipping FOREIGN KEYS:** Results in "Orphaned Records" when deleting.
- ✗ **No CHECK on business rules:** E.g., allowing negative bank balances or negative ages.



Best Practices for Data Integrity

- ✓ **Every table needs a Primary Key:** Even join tables usually benefit from one.
- ✓ **Use Foreign Keys rigorously:** Map out your ER diagram before writing SQL.
- ✓ **Use NOT NULL wisely:** Make it the default, allow NULL only when explicitly needed.
- ✓ **Push logic to the DB:** Use CHECK constraints for absolute business rules.
- ✓ **Name your constraints:** E.g., `CONSTRAINT chk_age CHECK (Age > 0)` makes debugging easier.

Summary: Integrity at a Glance



NOT NULL

No empty values
allowed.



UNIQUE

No duplicate values
allowed.



PRIMARY KEY

Unique row
identifier.



FOREIGN KEY

Table relationship
link.



CHECK

Validates boolean
conditions.



REFERENTIAL

Prevents orphaned
records.



Quick Review Quiz

(Hover over the card to reveal the answer)

1. Which constraint prevents duplicate values?



2. Can a PRIMARY KEY be NULL?



3. Which constraint connects two tables?



4. Which constraint validates Age ≥ 18 ?



5. What is Referential Integrity?



Great Job!